Hackaton Talento Tech

Participantes:

Jeronimo Tabares Gallego
Samir Steven Avendaño Bolaños
Marlon Andres Velasquez Umaña

Repositorio:
https://github.com/jetabaresg/crypto.git

28/3/2026

Hackathon 2026

Grupo: Crypto

Autores: Jerónimo Tabares Gallego- Samir Steven Avendaño- Marlon Andres Velasquez

1. Resumen Ejecutivo

CryptoScan es una solución automatizada de auditoría criptográfica desarrollada para evaluar la postura de seguridad en la capa de transporte (TLS/SSL) de infraestructuras web.

- **Herramientas Utilizadas:** La herramienta está construida íntegramente en Python 3 (usando únicamente la librería estándar para evitar dependencias externas) y actúa como un orquestador que paraleliza y unifica la ejecución de cuatro motores de escaneo de la industria: Nmap, TestSSL.sh, SSLScan y OpenSSL.
- **Estructura del Código:** El proyecto se diseñó bajo una arquitectura modular y escalable, dividida en cuatro bloques principales:
    - main.py: Punto de entrada (CLI), manejo de argumentos y orquestador del escaneo por lotes.
    - motores/: Contiene la lógica de ejecución de subprocesos (analizadorTLS.py) y el sistema inteligente de calificación de riesgos (evaluador.py).
    - parsers/: Módulos basados en Expresiones Regulares (Regex) dedicados a interpretar y normalizar la salida de texto crudo de cada herramienta específica.
    - report/: Generador del código HTML sin dependencias externas (generator.py).

El ciclo de la herramienta concluye proponiendo un plan de remediación táctico. **De manera innovadora, este plan de mitigación es generado dinámicamente mediante la integración**

de un Modelo de Lenguaje Grande (LLM) vía API, proporcionando recomendaciones contextuales y específicas para la topología de fallos encontrada en cada servidor analizado.

2. Metodología de Auditoría

La metodología de CryptoScan se basa en un enfoque de **"Confirmación Multi-Motor"**. En lugar de depender de una sola herramienta (que podría arrojar falsos negativos o fallar), el sistema lanza sondas concurrentes utilizando Nmap, TestSSL, SSLScan y comandos puros de OpenSSL.

El proceso sigue estos tres pasos fundamentales:

1. **Enumeración:** Se envían paquetes ClientHello manipulados para obligar al servidor a revelar todos los protocolos y suites de cifrado que está dispuesto a aceptar.
2. **Extracción y Normalización:** Las salidas crudas de las 4 herramientas son capturadas mediante subprocess. Los módulos parsers aplican expresiones regulares para extraer versiones exactas de TLS y nombres de algoritmos (ej. aislando "RC4" o "AES-GCM"), eliminando duplicados para crear una base de datos limpia.
3. **Ponderación de Riesgo:** El motor de evaluación parte de un Score perfecto de 100. Por cada hallazgo vulnerable detectado en la lista normalizada, el sistema aplica penalizaciones matemáticas (Crítico: -40pts, Alto: -25pts, Medio: -15pts, Bajo: -8pts) para calcular el nivel de riesgo final.

3. Análisis de Resultados (Preguntas Clave)

- **¿Qué versiones de TLS están habilitadas en cada servidor?**
  Durante nuestras pruebas, identificamos que servidores modernos (ej. google.com) operan exclusivamente con TLS 1.2 y TLS 1.3. Sin embargo, en el entorno vulnerable evaluado (34.45.64.235), detectamos la habilitación activa de protocolos altamente depreciados, específicamente SSLv3, TLS 1.0 y TLS 1.1.
- **¿Qué servicios siguen permitiendo configuraciones obsoletas o inseguras?**
  El servicio web operando en el puerto 443 del servidor objetivo (34.45.64.235) permite configuraciones inseguras al aceptar negociaciones con algoritmos de cifrado simétrico considerados criptográficamente rotos, tales como RC4, 3DES, DES y MD5, así como cifrados nulos (NULL) y de grado de exportación (EXPORT).

- **¿Cuáles hallazgos representan mayor criticidad?**
  La criticidad máxima recae en el soporte de suites como **RC4 y 3DES**. Estos algoritmos carecen de Secreto Hacia Adelante (PFS) y son vulnerables a ataques de fuerza bruta, colisiones y ataques de texto cifrado (como SWEET32). Permitir **SSLv3 y TLS 1.0** también es crítico, ya que expone al servidor a ataques de degradación (downgrade attacks) como POODLE o BEAST, permitiendo la intercepción de tráfico (Man-in-the-Middle).
- **¿Qué acciones deben ejecutarse para fortalecer la configuración?**
  - Modificar la configuración del servidor web (ej. Nginx/Apache) para forzar estrictamente el uso de TLS 1.2 y TLS 1.3.
  - Depurar la lista de cipher suites, eliminando RC4, 3DES y algoritmos basados en MD5, priorizando aquellos con curvas elípticas (ej. ECDHE-RSA-AES256-GCM-SHA384).
  - Implementar cabeceras de seguridad HSTS (HTTP Strict Transport Security) para evitar conexiones HTTP no cifradas.
- **¿Cómo comparar el nivel de exposición entre varios dominios o entornos?**
  Nuestra herramienta resuelve este problema de manera nativa mediante su funcionalidad de procesamiento por lotes. Al ingresar varias IP , CryptoScan analiza los entornos secuencialmente y genera una **Vista Consolidada en Terminal**. Esta tabla resume y compara el Nivel de Riesgo (Crítico, Alto, Medio, Bajo) y el Score exacto de cada IP, permitiendo a los equipos de infraestructura priorizar qué servidor parchear primero.

4. Diseño de la Solución End-to-End (Integración de Componentes)

CryptoScan fue diseñado estrictamente siguiendo el flujo del reto, integrando de manera fluida los componentes requeridos en la rúbrica:

Componente 1: Ingreso y Validación de Objetivos
Diseñamos un sistema robusto "Anti-caídas". A través del módulo argparse, la herramienta acepta un objetivo individual o listas separadas por espacios.

- **Validación Lógica:** Usamos ipaddress y Regex para asegurar que el input sea una IP o dominio válido.

- **Validación de Red (Manejo de Errores):** Antes de lanzar herramientas pesadas, implementamos una prueba de conexión por sockets nativa de Python con un timeout de 3 segundos. Si el puerto 443 está cerrado o el dominio no existe, el sistema reporta el error limpiamente y salta al siguiente objetivo sin congelarse ni cerrarse abruptamente (evitando excepciones críticas del sistema).

Componente 2: Motor de Análisis TLS
Este es el núcleo de recolección de datos. Orquestamos la ejecución de sslscan, nmap, testssl.sh y openssl mediante el módulo subprocess de Python, pasándoles el objetivo validado.

- Implementamos Timeouts estrictos (25 segundos por herramienta) para evitar bucles infinitos.
- Desarrollamos una suite de Parsers personalizados. Cada herramienta tiene un parser dedicado que extrae únicamente la información de valor (versiones de protocolos y nombres de cifrados), desechando la "basura" visual de la terminal.

Componente 3: Motor de Evaluación y Recomendaciones Asistido por IA
Los datos limpios extraídos por el Componente 2 pasan a nuestro motor de reglas y análisis heurístico (evaluador.py).

- **Clasificación de Riesgo (Scoring):** Creamos un sistema de Scoring ponderado. Dependiendo de las vulnerabilidades encontradas (ej. presencia de "RC4" o "SSLv3"), el sistema resta puntos de un total de 100, asignando el nivel de riesgo final: Crítico, Alto, Medio o Bajo.
- **Generación Dinámica de Recomendaciones vía IA:** En lugar de depender exclusivamente de recomendaciones estáticas, el sistema fue actualizado con un módulo de IA (AI Recommender). Utilizando la API de OpenRouter, el sistema envía de forma segura el contexto de las vulnerabilidades encontradas a un Modelo de Lenguaje Grande (LLM) — específicamente openai/gpt-4o-mini. La IA analiza la combinación exacta de fallos del servidor y genera un plan de mitigación personalizado, detallado y accionable.
- **Seguridad de Credenciales:** La conexión con la API se gestiona de forma segura a través de variables de entorno (AI_RECOMMENDER_API_KEY,

OPENROUTER_API_KEY), garantizando que ninguna clave sensible quede expuesta en el código fuente.

TABLA I. HALLAZGOS TÉCNICOS Y VULNERABILIDADES DETECTADAS

Severidad	**Vulnerabilidad Identificada**	**Descripción Técnica**
CRÍTICA	Soporte de Suites de Cifrado Obsoletas	El servidor negocia algoritmos criptográficamente rotos (RC4, 3DES, MD5). Estos carecen de Secreto Hacia Adelante (PFS) y son susceptibles a ataques de colisión y fuerza bruta [1].
ALTA	Versiones SSL/TLS Depreciadas	Habilitación de SSLv3, TLS 1.0 y TLS 1.1. Protocolos prohibidos por el IETF debido a fallas de diseño inherentes susceptibles a ataques como POODLE o BEAST [2].

Anexo: Glosario de Siglas y Términos Técnicos

A continuación, se detallan las siglas y conceptos técnicos utilizados a lo largo de este documento para facilitar la comprensión de la arquitectura y los hallazgos de **CryptoScan**:

- **API (Application Programming Interface):** Interfaz de Programación de Aplicaciones. Es el puente que permite a CryptoScan comunicarse de forma segura con los servicios externos, como el modelo de Inteligencia Artificial (OpenRouter), para enviar datos y recibir las recomendaciones.
- **CLI (Command Line Interface):** Interfaz de Línea de Comandos. Entorno basado en texto a través del cual el usuario interactúa con nuestra herramienta, ejecutando las órdenes de escaneo directamente en la terminal.

- **DES / 3DES (Data Encryption Standard / Triple DES):** Estándares de cifrado simétrico desarrollados en los años 70 y 90. Actualmente son considerados criptográficamente débiles, obsoletos y vulnerables a ataques modernos.
- **HSTS (HTTP Strict Transport Security):** Política de seguridad web que obliga a los navegadores a comunicarse con el servidor web únicamente a través de conexiones seguras (HTTPS), previniendo ataques de intercepción.
- **HTML (HyperText Markup Language):** Lenguaje de Marcado de Hipertexto. Es el formato estándar utilizado por el módulo generator.py para construir el reporte visual final de la auditoría.
- **IETF (Internet Engineering Task Force):** Grupo de Trabajo de Ingeniería de Internet. Es la organización internacional principal responsable de desarrollar y promover los estándares de Internet (por ejemplo, son quienes emitieron la prohibición oficial de usar TLS 1.0 y TLS 1.1).
- **IP (Internet Protocol):** Protocolo de Internet. La dirección numérica o etiqueta que identifica a cada servidor o dispositivo analizado en la red (ej. 34.45.64.235).
- **LLM (Large Language Model):** Modelo de Lenguaje Grande. Es el tipo de Inteligencia Artificial que alimenta nuestro motor de recomendaciones (ej. GPT-4o-mini). Está entrenado con cantidades masivas de texto para entender el contexto técnico y generar planes de mitigación redactados de forma natural.
- **MD5 (Message-Digest Algorithm 5):** Algoritmo de reducción criptográfica (hash) creado en 1991. Hoy en día está roto y es vulnerable a "ataques de colisión", por lo que su uso en certificados y suites de cifrado es un riesgo crítico.
- **PFS (Perfect Forward Secrecy):** Secreto Hacia Adelante (o Secreto Perfecto Hacia Adelante). Propiedad de seguridad criptográfica que garantiza que, incluso si un atacante roba la clave privada del servidor en el futuro, no podrá descifrar el tráfico que haya sido capturado en el pasado.
- **RC4 (Rivest Cipher 4):** Algoritmo de cifrado de flujo muy popular en el pasado, pero actualmente prohibido debido a múltiples vulnerabilidades que permiten descifrar el tráfico en tiempo real.
- **Regex (Regular Expressions):** Expresiones Regulares. Secuencias lógicas de caracteres que forman patrones de búsqueda. CryptoScan las utiliza intensivamente en sus parsers para extraer versiones y nombres exactos de protocolos del texto desordenado de la terminal.

- **SSL (Secure Sockets Layer):** Capa de Conexiones Seguras. Es el predecesor histórico de TLS. Sus versiones (como SSLv2 y SSLv3) tienen fallas de diseño estructurales irreparables (como la vulnerabilidad POODLE) y nunca deben estar habilitadas.
- **TLS (Transport Layer Security):** Seguridad de la Capa de Transporte. Es el protocolo criptográfico estándar moderno diseñado para proteger la privacidad y la integridad de los datos que viajan por Internet. (Las versiones seguras recomendadas son TLS 1.2 y TLS 1.3).

Plantilla

3. DESCRIPCIÓN GENERAL DE LA HERRAMIENTA (MÁX. 100 PALABRAS)
¿Qué hace su herramienta? ¿Qué tipo de análisis realiza? ¿Cómo ayuda a identificar riesgos de seguridad?

CryptoScan es un orquestador automatizado en Python que evalúa la seguridad de la capa de transporte (TLS/SSL) en servidores web. Realiza un análisis multi-motor integrando Nmap, TestSSL, SSLScan y OpenSSL para extraer las versiones de protocolos y suites de cifrado activas. Ayuda a identificar riesgos críticos al detectar configuraciones obsoletas (como RC4 o SSLv3), asignando un nivel de riesgo ponderado (Score) y generando planes de remediación dinámicos asistidos por Inteligencia Artificial (LLM) para prevenir ataques de interceptación. *(78 palabras)*

4. FUNCIONALIDADES PRINCIPALES

Funcionalidad / Componente	Descripción breve	Nivel de implementación	Evidencia / observaciones
Ingreso de objetivos	Permite ingresar dominios o direcciones IP, ya sea individualmente o por lotes	100% (Completado)	Implementado vía argparse en main.py.

Validación de entrada	Verifica formato (Regex) y realiza un pre-chequeo de red (sockets).	100% (Completado)	Módulo analizadorTLS.py con manejo de errores anti-caídas.
Análisis de versiones TLS	Identificación de protocolos habilitados mediante ejecución concurrente de sondas.	100% (Completado)	Integración de 4 herramientas usando subprocess.
Detección de configuraciones inseguras	Identifica versiones obsoletas o débiles en texto crudo de terminal.	100% (Completado)	Uso intensivo de *Parsers* y Expresiones Regulares (Regex).
Clasificación de riesgo	Asigna nivel de criticidad (Score 0-100) a los hallazgos.	100% (Completado)	Lógica de penalizaciones en evaluador.py.
Generación de recomendaciones	Propone acciones de remediación tácticas y específicas.	100% (Completado)	Integración de API LLM (OpenRouter/GPT) para planes dinámicos.
Reporte de resultados	Presenta resultados de forma clara en archivos y terminal.	100% (Completado)	Reportes HTML generados en generator.py y Vista Consolidada en CLI.

5. TECNOLOGÍAS Y ARQUITECTURA DEL SISTEMA

Componente	Tecnología / Servicio	Descripción de uso en el proyecto
Lenguaje principal	Python 3	Orquestación del flujo, *parsing* de datos y lógica del sistema (usando solo librería estándar, 0 dependencias).
Motor de análisis TLS	OpenSSL, testssl.sh, nmap, sslscan	Escaneo concurrente de puertos y enumeración exhaustiva de cifrados.
Validación de objetivos	socket, ipaddress, re (Regex)	Pre-validación estricta de formato y conectividad antes de lanzar escaneos pesados.
Motor de clasificación de riesgo	Reglas propias (Python) + API OpenRouter (LLM)	Ponderación matemática de vulnerabilidades y generación contextual de recomendaciones con IA.
Generación de reportes	HTML generado vía Python, CLI	Creación de entregables visuales en HTML estático y tabla resumen en consola.
Interfaz de usuario	CLI (Línea de Comandos)	Interacción nativa para facilitar la automatización y el análisis por lotes (Batch).
Infraestructura / ejecución	Local	Ejecución directa y segura en la máquina del analista sin exponer datos a terceros.

6. INDICADORES DE DESEMPEÑO (KPIs)

Indicador	Fórmula / Método de medición	Resultado obtenido / Meta	Interpretación
Objetivos analizados	Número de dominios o IPs evaluados correctamente.	Meta: >1	El sistema soporta la carga y análisis por lotes sin interrupciones por error.
Versiones TLS detectadas	Total de protocolos identificados por servidor.	Meta: 100% precisión	Los *parsers* lograron extraer correctamente desde protocolos obsoletos (SSLv3) hasta seguros (TLS 1.3).
Hallazgos inseguros detectados	Número de configuraciones obsoletas o inseguras encontradas.	Meta: >0 (en entornos vulnerables)	Identificación precisa de cifrados rotos como RC4, 3DES o MD5 en el nodo objetivo.
Tiempo de análisis	Promedio en segundos por objetivo evaluado.	Meta: < 30s por objetivo	Optimizado mediante ejecución en paralelo y *timeouts* preventivos por sonda.
Recomendaciones generadas	Número de acciones de remediación propuestas.	Meta: 3+ acciones precisas	La IA generó un plan de acción acorde a la criticidad real de las métricas obtenidas.

7. ENLACES Y EVIDENCIAS

Recurso	URL / Ubicación

Repositorio del proyecto (GitHub)	https://github.com/jetabaresg/crypto
Herramienta o demo funcional	
Notebook o script principal	main.py (dentro del repositorio GitHub)
Reporte de resultados	Archivo reporte_tls_34.45.64.235.html
Presentación de Pitch	

8. RESULTADOS Y CONCLUSIONES

Principales logros técnicos:

1. Diseño de una arquitectura multi-motor altamente modular utilizando únicamente librerías estándar de Python, garantizando portabilidad sin depender de pip install.
2. Integración exitosa de un Modelo de Lenguaje Grande (IA) mediante API, transformando reportes estáticos en consultoría de seguridad dinámica y contextualizada.
3. Creación de un sistema "Anti-caídas" y procesamiento por lotes, permitiendo analizar múltiples servidores y generar vistas consolidadas sin que el script se cierre por errores de red.

Desafíos enfrentados:

El mayor reto fue procesar e interpretar las salidas de texto ("salida cruda" o *stdout*) de 4 herramientas de terminal distintas, las cuales no tienen formatos estandarizados. Además, gestionar los *timeouts* para evitar que el script se colgara indefinidamente al toparse con IPs inexistentes.

Lecciones aprendidas:

Dominio en la implementación avanzada de Expresiones Regulares (*Regex*) para la limpieza de datos (*parsing*). Asimismo, comprendimos la importancia de orquestar subprocesos (subprocess en Python) de forma segura y aplicar buenas prácticas en la gestión de credenciales (variables de entorno para la API Key de la IA).

Resultados

```
└─$ ls
main.py  motores  parsers  __pycache__  README.md  report  reporte_tls.html  requirements.txt  scripts

┌──(kali㉿kali)-[~/crypto]
└─$ python main.py 34.45.64.235
[+] Objetivo: 34.45.64.235:443
[+] Recomendaciones: fallback local
[+] Riesgo: Alto | Score: 50/100 (4/4 validas, 0 invalidas)
[+] Score base: 50 | Penalizacion por cobertura: -0
[+] Estado de prueba: Prueba realizada correctamente
[+] Reporte generado: /home/kali/crypto/reporte_tls.html

┌──(kali㉿kali)-[~/crypto]
└─$
```

Reporte de Seguridad TLS

Objetivo: 34.45.64.235:443

Score: 50/100 (4/4 validas, 0 invalidas)

Estado de prueba: Prueba realizada correctamente

Riesgo: Alto

Protocolos detectados: TLSv1, TLSv1.0, TLSv1.1, TLSv1.2, TLSv1.3

Resumen de Severidades

0	2	0	0
Critical	High	Medium	Low

Hallazgos

Severidad	Titulo	Detalle	Recomendacion
high	Versiones TLS/SSL inseguras habilitadas	Se detectaron versiones obsoletas: TLSv1, TLSv1.0, TLSv1.1	Deshabilitar SSLv2/SSLv3/TLS 1.0/ TLS 1.1 y permitir solo TLS 1.2 y TLS 1.3.
high	Cifrados debiles detectados	Se detectaron suites debiles: AECDH-NULL-SHA Curve 25519 DHE 253, ECDHE-RSA-NULL-SHA, ECDHE-RSA-NULL-SHA Curve 25519 DHE 253, NULL-MD5, NULL-SHA, NULL-SHA256	Eliminar suites debiles y priorizar AES-GCM o CHACHA20 con ECDHE.

Estado de Herramientas

Herramienta	Estado	Protocolos	Cifrados	Detalle
nmap	OK	TLSv1.0, TLSv1.1, TLSv1.2	34	-
testssl	OK	TLSv1, TLSv1.1, TLSv1.2	16	-
sslscan	OK	TLSv1.0, TLSv1.1, TLSv1.2	34	-
openssl	OK	TLSv1, TLSv1.1, TLSv1.2, TLSv1.3	0	-

Recomendaciones

- Habilitar solamente TLS 1.2 y TLS 1.3.
- Deshabilitar suites de cifrado obsoletas o debiles.
- Mantener OpenSSL, servidor web y librerias criptograficas actualizadas.
- Aplicar HSTS y revisar configuraciones periodicamente.
- Deshabilitar SSLv2/SSLv3/TLS 1.0/TLS 1.1 y permitir solo TLS 1.2 y TLS 1.3.
- Eliminar suites debiles y priorizar AES-GCM o CHACHA20 con ECDHE.

Nuestro reporte documenta los protocolos utilizados y las vulnerabilidades identificadas en una o más direcciones ips o dominios , mediante las herramientas empleadas. Se realizó una evaluación de la criticidad de cada hallazgo y, con base en los datos recopilados, se generaron recomendaciones utilizando inteligencia artificial a través de una API key. Finalmente, se estableció una puntuación del estado de seguridad, la cual disminuye en función de la cantidad de vulnerabilidades detectadas y su nivel de criticidad.